# Operators and Expressions

## ✱ Objective

— To understand different operators available in C language

— To understand how expressions are evaluated.

## ✱ Operators

— What is operator? What is operand?

3 + 7 — operand (3), operator (+), operand (7)

+7 , −8 — unary operator

— Types of operators (Based on number of operands)

- Unary — operates on one operand (+, −, !, ++, --)
- Binary — " " two " (+, −, *, &&, <...)
- Ternary — " " three " (?:)

— Other classification (Based on what operator does)

1. Arithmetic operators (+, −, *, /, %)
2. Relational operators (>, >=, <, <=, ==, !=)
3. Logical operators (&&, ||, !)
4. Assignment operators (=, +=, -=, *=, /=, %=)
5. Increment and decrement operators (++, --)
6. Conditional operator (ternary operator) ?:
7. Bitwise operators (bitwise logical (&, |, ^), <<, >>, ~
8. special operators. (comma operator, sizeof)

3+7

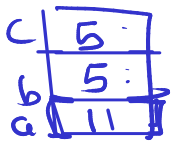
3>7 (0) false

3<7 (1) true

| a | b | a&&b | a\|\|b | !a | !b |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | -17 | 0 | 1 | 1 | 0 |
| -1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 |

true

```
int a, b, c;
a = 5;
b = c = a;
```




② ①

a += 5; // a = a + 5;

= (5 + 6)

7 += b; Invalid. LHS must be variable
7 = b;

## 5. Increment and decrement operators

```
int num = 7;
num++;   // num = num + 1;   | num = 8
++num;   // num = num + 1;   | num = 9
```

post-increment →
pre-increment

(++) - one operator

```
int num2;
num2 = num++;   // num2 = num; num += 1;
```
assign and then increment

num = 10, num2 = 9

```
num2 = ++num;   // num += 1; num2 = num;
```
increment first and then assign

num = 11, num2 = 11

⇒ Try on your own

① 7++;
++7;
int x = 8++;

} Valid? Not valid

a++;
a = a + 1;

7++;
7 = 7 + 1;
×

② float f = 9.7;
f++;
++f;
} Valid? Yes

---

```
int m = 7, n;
```

pre-decrement → n = --m; // m = 6, n = 6

```
m -= 1;
n = m;
```

post-decrement → n = m--; // m = 5, n = 6

```
n = m;
m -= 1;
```

## 6. Conditional operator (Ternary operator)

```
int a;
7 > 5 ? a = 1 : a = 2;
```

operand | operand (if true) | operand (if false)

one operator

a would be 1.

```
int a, b, c;
a = 3, b = 5, c = 7;
(3 > 5)
a > b ? a-- : b--;
```

a = 3, b = 4, c = 7

---

```
int r = 9, s = 12, t;
t = r > s ? r : s;
        (12)
```

r = 9
s = 12
t = 12

finding maximum of r and s and storing it in t.

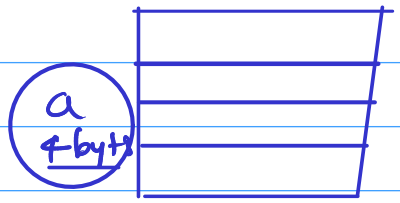
```
t = r > s ? s : r;
t = r < s ? r : s;
```

smaller of r and s is stored in t.

# 7. Bitwise operators

1. Bitwise logical operators ✓
2. Bitwise shift "
3. one's complement operator

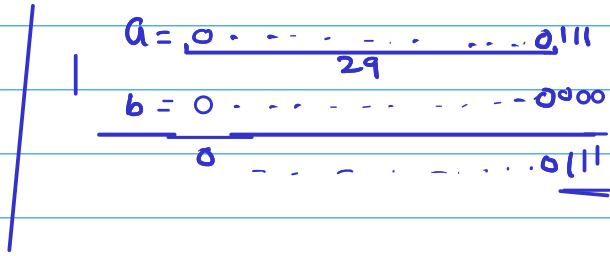
## 1. Bitwise logical operators




```
int a=7, b=0;
```

Logical operators:
- `a && b` // 0
- `a || b` // 1

Bitwise logical operators:
- `a & b` // 0
- `a | b` // 7




```
int p=10, q=20, r;
```

| | | | |
|---|---|---|---|
| `r = p && q;` | p=10 | q=20 | r=1 |
| `r = p \|\| q;` | p=10 | q=20 | r=1 |
| `r = p & q;` | p=10 | q=20 | r=0 |
| `r = p \| q;` | p=10 | q=20 | r=30 |

00 0 1 0 1 0
00 1 0 1 0 0
00 1 1 1 1 0
128 64 32 16 8 4 2 1
16 + 8 + 4 + 2 = 30

① Bitwise and (&)
② Bitwise or (|)
③ Bitwise xor (^) exclusive or

one bit

| | | & | \| | ^ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 |

```
int a=12, b=17, c;

...000|100
...00|1001
-------
    |1101

c = a & b;  // a=12  b=17  c=0
c = a | b;  // a=12  b=17  c=29
c = a ^ b;  // a=12  b=17  c=29
```

---

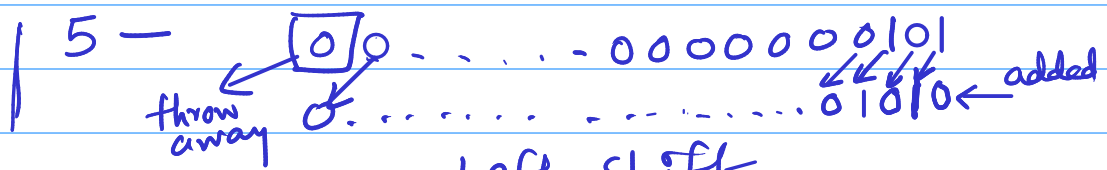
```
int x=7, y=29, z;

...000|111
...0|1110|1 ^
-------
  0|1010

z = x & y;  // x=7  y=29  z=5
z = x | y;  // x=7  y=29  z=31
z = x ^ y;  // x=7  y=29  z=26
```

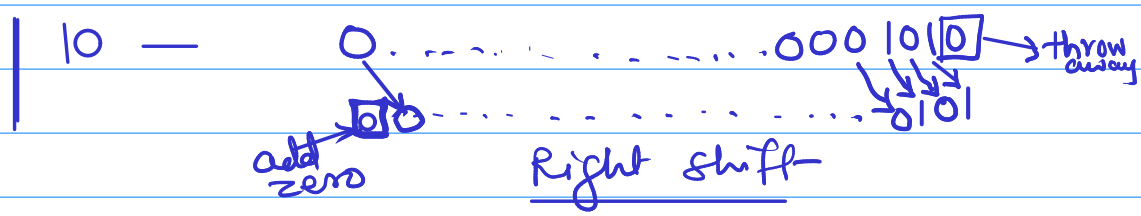
---

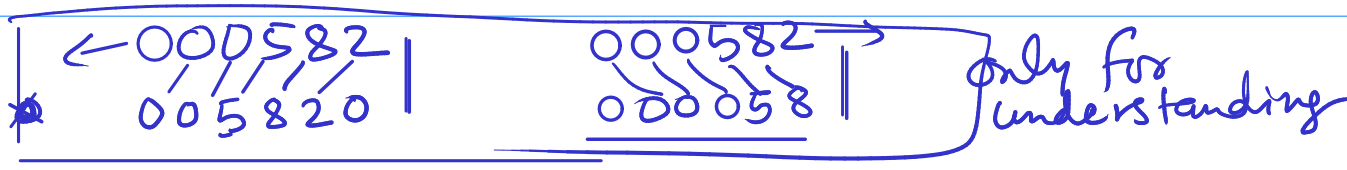
⇒ No bitwise operation on float or double.

```
int t = 5 ^ 9;   ✓
t = 7 | 5;       ✓
8 & 4;   valid   But it has no impact.
```

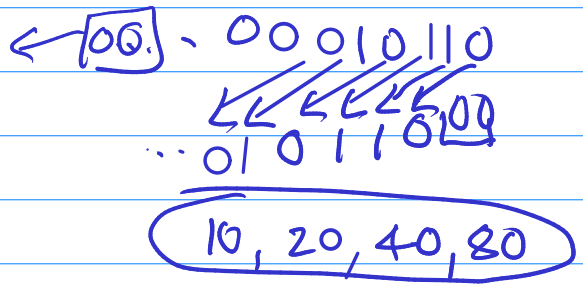
## 2. Bitwise shift operators



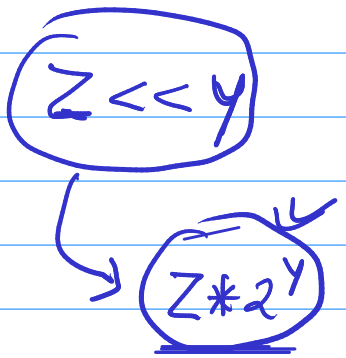
```
int x=7;

x<<1;  // x=7
x+1;   // x=7

x = x<<1; // x=14
x = x<<2; // x=56
```




```
int z=7;

z = z>>1; // z=3

z=98;

z = z>>3; // z=12
```

49, 24, 12

$O_{2\times2\times2}$


3. one's complement operator → converts 0 to 1 and 1 to 0 in bits.

```
int x=5;
x=~x;
```

↳ complement operator

000...000101 x

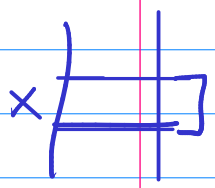
111...111010 ~x

int long x = 7;

x = x << 7; ✓

## 8. Special operators

### 1. sizeof operator —— sizeof( )



```
int x;
int sz = sizeof(x); // size of int on your compiler is 4. sz = 4.
```

```
sz = sizeof(7); // sz = 4.
     (int)
sz = sizeof(3.4); // Most systems sz = 8
```

↳ double

Programming in ANSI C. – E Balagurusamy — Book

### 2. Comma operator

```
int a=5, b=7, c;
c = a+b;
a = 3;
```

a=3 b=7

c=12

```
int a=3, b=5, c=7, d;
d = a, c, b; // d=5   a=3 b=5 c=7
    a+c+b
```

```
d = a=c, c=b, a+c;
```

a=7 b=5 c=5 d=12

```
int a=5, b=7, tmp;

tmp=a;
a=b;
b=tmp;

tmp=a, a=b, b=tmp;
```

## ☆ Expression

3+7

```
int x=3;
x+2; //x=3
```

`X = x+2` valid expression
Not a statement

`X=x+2;` valid expression
& valid statement

→ Integer arithmetic

```
int x, y, z;
x=2;
x=x+3;
z=2;
y=3;
x=x+y+2*z;
y=y/z;
```

1.5 or 1

```
float f=y/z;   //f=1.000000
```

int

→ Mixed mode arithmetic → `f=(float)y/z;  //f=1.5`

→ floating-point arithmetic

```
float f = 7.2, g = 9.1;
float h = (f * g);
```

h ← float

→ `char c = 5;` `int i;` `char d = 'p';`

d ← Ascii of 'p'

`i = c * 3;` ✓ valid // i = 15

c → convert to int

→ char is converted to int for expression evaluation

→ Mixed mode arithmetic

| char | int<br>long int<br>long long int<br>unsigned int<br>unsigned long int<br>unsigned long long int | float<br>double<br>long double |
|---|---|---|

Note → Never mix signed & unsigned variables/constants in an expression

① char is converted to int

② If expression has long double then result would be long double

```
long double x=7.5;
double y=7.3;
long long int z=3958;
int z= x*y+z
```

int ← long double

does not have long double

③ If expression has double then result is double

```
int x=7;
float f=8.5;
x*f/3.2
```

double → double

does not have long double and double

④ If expression has float then result is float

```
float f=6.2;
int y=3;
f*y
```

→ float

⑤ long long → long long

⑥ long → long

⑦ int → int

⑧ char → int

→ Implicit & explicit conversion

```
int i = 7.3;
```

int ← double // implicit (automatically) conversion

```
int i = (int) 7.3;
```

int ← int

explicitly telling to convert to int

→ explicit conversion

```
double x = (int) 7.3;
```

7

// x = 7.00000

## ★ Evaluation of expression

3+5*2 // 13 ✓ or 16

3*5/2 // 7 ✓

→ precedence : priority of that operator
of operators

✓ → Associativity : when operators have same precedence
of operators left to right or right to left

✓ 3*5+7

3+5*7

✓

3*7*5/3

*, / → same precedence

3*7/5

→ 21/5

+, − same precedence

---

3*7 + 4*8

((3*7) + (4*8)) ✓

✓ ✓

3*(5/2) // 6

```
int a=5, b=7;

int c = a = b; // a=7  b=7  c=7
```

(c = (a = b))

→ what has more importance precedence or associativity?

→ precedence ✓